AES-128 AXI4-Lite Accelerator IP

Architecture, Memory Mapping & Slave Memory Design

Overview

This project implements a complete AES-128 encryption accelerator integrated with AXI4-Lite interfaces. The design operates as a hardware slave peripheral accessible by a CPU through standard AXI4-Lite register reads and writes. Internally, it maintains a 10-stage pipelined datapath that processes 128-bit plaintext blocks through all AES encryption rounds, while dedicated memory controllers manage bulk plaintext reads from external memory and bulk ciphertext writes back. The accelerator demonstrates how to cleanly separate control (register interface), data transformation (crypto pipeline), and memory subsystem concerns in a reusable IP block.

High-Level Architecture

The AES-128 accelerator consists of four main functional domains:

Control Domain:

Handles CPU-facing AXI4-Lite register operations. The controller_interface manages read/write transactions on the CPU-facing AXI port, decoding addresses and forwarding register read/write requests to the internal controller. The controller maintains the register file and generates start/stop signals and configuration values (encryption key, memory boundaries).

Key Expansion Domain:

Precomputes all ten 128-bit round keys from the user-supplied static AES-128 key. The key_scheduler module expands the 128-bit master key into eleven 128-bit round keys (including the original key as round_key_0) using the standard AES key schedule algorithm with S-box lookups. All round keys are made available combinationally to the datapath, eliminating latency during encryption.

Memory Domain:

Manages bulk data transactions with external memory. Two independent paths: the read controller reads plaintext blocks from external memory starting at start_address up to end_address, storing 32-bit words into the read_mem FIFO. The write controller reads encrypted blocks from the write_mem FIFO and writes them back to external memory at corresponding addresses.

Encryption Domain (Datapath):

A 10-stage pipeline that processes plaintext through the AES-128 algorithm. Each stage implements one complete AES round (SubBytes, ShiftRows, MixColumns, AddRoundKey). The final stage omits MixColumns

as per AES standard. Valid_in and valid_out signals handshake data into and out of the pipeline, enabling smooth stalling when memory buffers are full.

Slave Memory: Register Map & Addressing

The accelerator exposes an 8-register control interface to the CPU via a 3-bit address bus (3'h0 through 3'h7). Each register is 32 bits wide. AXI4-Lite reads and writes are decoded by the controller_interface and forwarded to the controller's internal register file.

Register	Address	R/W	Purpose	Remarks
CTRL	3'h0	RW	Control & Start	Bit [0]: START signal. Writing 1 to this register initiates encryption.
STATUS	3'h1	RO	Status	0=IDLE, 1=BUSY, 2=ERROR, 3=DONE. Automatically transitions; re
START_ADDR	3'h2	RW	Start Address	32-bit base address for plaintext reads in external memory. Must be v
END_ADDR	3'h3	RW	End Address	32-bit end address (exclusive) for plaintext reads. Controls how much
KEY[0]	3'h4	RW	AES Key Word 0	Bits [31:0] of the 128-bit AES key. KEY = {KEY[3], KEY[2], KEY[1], K
KEY[1]	3'h5	RW	AES Key Word 1	Bits [63:32] of the 128-bit AES key.
KEY[2]	3'h6	RW	AES Key Word 2	Bits [95:64] of the 128-bit AES key.
KEY[3]	3'h7	RW	AES Key Word 3	Bits [127:96] of the 128-bit AES key.

Write Constraints & State Machine

The controller implements a state machine to protect against race conditions. Registers cannot be written arbitrarily at any time:

CTRL Register (3'h0): Can only be written when STATUS is IDLE. Once the START bit is written, the STATUS transitions to BUSY. The START bit is automatically cleared when encryption completes (DONE or ERROR). Attempting to write to CTRL while BUSY has no effect.

STATUS Register (3'h1): Read-only. The controller manages transitions internally: IDLE → BUSY → (DONE or ERROR) → IDLE. All other registers can be written anytime except CTRL (which is protected during BUSY state).

KEY[0-3] Registers (3'h4-3'h7): Should be written before initiating encryption. Once START is asserted, keys are latched and used immediately. Writing new keys during BUSY does not affect the ongoing encryption.

Typical Software Usage Flow

1. Configure Start/End Addresses

Write the base address of plaintext data to START_ADDR (3'h2). Write the end address (exclusive) to END_ADDR (3'h3). Data between these addresses will be encrypted block-by-block.

2. Load Encryption Key

Write four 32-bit words to KEY[0] through KEY[3] (3'h4-3'h7) to set the 128-bit AES key. The order is little-endian: KEY[0] is the LSB, KEY[3] is the MSB.

3. Start Encryption

Write any value (typically 1) to CTRL (3'h0), bit [0]. The STATUS immediately transitions to BUSY.

4. Poll for Completion

Poll STATUS (3'h1) until it is no longer BUSY. STATUS = 3 (DONE) indicates successful completion; STATUS = 2 (ERROR) indicates a memory access error.

5. Retrieve Ciphertext

Encrypted blocks are written to external memory starting at start_address. Read results back from memory at the same addresses.

Slave Memory System Design

The accelerator features a dual-FIFO architecture for memory buffering. Unlike a simple register interface, the accelerator reads multiple plaintext blocks from external memory and queues them for processing, then drains processed results back to memory. This asynchronous design decouples memory latency from encryption throughput.

Read Memory (Input FIFO)

Purpose: Buffers plaintext blocks fetched from external memory before entering the encryption pipeline.

Architecture: A 4-entry circular FIFO implemented with dual-pointer logic (write_ptr and read_ptr). Each FIFO entry is 64 bits: upper 32 bits store the block address, lower 32 bits store the data word. The FIFO uses a 3-bit pointer (with wrap-around at 4 entries), allowing the pointers to be 3 bits (0–7) to detect full/empty states.

Full/Empty Flags:
• **Empty:** read_ptr == write_ptr (pointers match exactly)
• **Full:** read_ptr[1:0] == write_ptr[1:0] AND read_ptr[2] != write_ptr[2] (low 2 bits match, but MSB differs, indicating a wrap-around)

Write Port (from read_controller): The read_controller writes 32-bit words and their source addresses. Every write increments write_ptr. The FIFO stalls if full (read_controller pauses AXI read requests).

Read Port (to encryption pipeline): When FIFO reaches full (contains 4 entries), it outputs a 128-bit plaintext block (combining 4 consecutive 32-bit words) along with a 128-bit address block. The valid_in signal goes high, signaling the datapath that valid data is available. The read_ptr increments by 4 (draining all 4 entries in one transaction) to allow new data.

Write Memory (Output FIFO)

Purpose: Buffers encrypted blocks from the pipeline before sending them back to external memory.

Architecture: Identical 4-entry circular FIFO structure to the read memory, with same full/empty logic and 3-bit pointers.

Write Port (from encryption pipeline): The datapath writes a 128-bit ciphertext block and corresponding 128-bit address block when valid_out is high. This transaction fills all 4 FIFO entries at once and increments write_ptr by 4. The FIFO stalls the pipeline if empty (no room for new results).

Read Port (to write_controller): The write_controller reads one 32-bit word and its address at a time using rd_en. Each read increments read_ptr by 1, allowing the controller to interleave small AXI writes without waiting for all 4 words. The empty signal tells the controller when no more data is available.

Asymmetry by Design: The read side drains all 4 words at once (128-bit granularity for the pipeline), while the write side feeds words individually (32-bit granularity for AXI compatibility). This allows fine-grained pipelining of write transactions without requiring the entire 128-bit result to be ready at once.

Stall Mechanism & Flow Control

The write_controller asserts a global stall signal when either FIFO reaches a critical state. This synchronizes all domains (key_scheduler, datapath, read_controller, read_mem, write_mem) to halt when necessary:

Read-side stall: When read_mem is full, the read_controller cannot fetch more plaintext. Stall is asserted to prevent the pipeline from consuming data and the key scheduler from advancing. This back-pressure gives the write controller time to drain results from write_mem.

Write-side stall: When write_mem is empty (or about to underflow), the pipeline cannot output results. Stall is asserted to halt the pipeline and read controller, preventing the system from deadlocking with a full read buffer but no room to deposit results.

Typical Flow: Under normal operation, the pipeline runs smoothly without stalls. If the external memory is slow (read latency), plaintext builds up in read_mem, triggering stall to throttle reads. If the write controller is slow (small write window), results back up in write_mem, triggering stall to prevent pipeline overflow.

Data Flow Through the System

Understanding how data moves through the AES accelerator illuminates why the memory and pipeline architectures are designed this way:

CPU issues START:

CPU writes CTRL register (3'h0) with START bit set. Controller transitions to BUSY state.

Read controller fetches plaintext:

Read controller initiates AXI4-Lite read transactions starting at start_address. Each ARVALID/RVALID handshake brings one 32-bit word into the read_mem FIFO.

Read memory buffers & packs:

After 4 words arrive (one 128-bit block), read_mem combines them into a 128-bit plaintext and addresses. It sets valid_in high and feeds the pipeline.

Pipeline processes block:

Plaintext enters stage 0 (AddRoundKey with round_key_0). Each clock cycle, it progresses through stages 1–9 (full AES rounds). Stage 10 handles the final round (SubBytes, ShiftRows, AddRoundKey without MixColumns). Latches propagate the data and address alongside through all stages.

Result enters write memory:

After 10 cycles, ciphertext emerges from stage 10 with valid_out high. Write_mem captures the 128-bit block and address, distributing them across 4 FIFO entries.

Write controller drains & transmits:

Write controller reads one 32-bit word at a time from write_mem. For each word, it performs an AXI4-Lite write transaction (AWVALID/WVALID handshakes) to the corresponding address in external memory.

Completion & status:

Write controller monitors end_address. When the last ciphertext block has been written, it asserts done signal. Controller transitions STATUS to DONE, and CPU reads STATUS to confirm encryption finished.

Encryption Pipeline Architecture

The encryption domain is a 10-stage pipeline that implements the AES-128 algorithm. Each stage performs one complete AES round (except the final stage, which omits MixColumns).

Initial Round (Stage 0):

AddRoundKey with round_key_0. The plaintext is XORed with the first round key derived from the master encryption key.

Main Rounds (Stages 1–9):

Each stage implements: SubBytes (byte-wise S-box substitution), ShiftRows (row-wise cyclic shifts of the state matrix), MixColumns (column-wise linear transformation over GF(2^8)), and AddRoundKey (XOR with the corresponding round key). The S-box (substitution box) is implemented as a lookup table for fast hardware execution.

Final Round (Stage 10):

SubBytes, ShiftRows, AddRoundKey with round_key_10. Note: MixColumns is omitted in the final round per AES standard. This produces the 128-bit ciphertext.

Key Expansion & Precomputation

The key_scheduler module precomputes all 11 round keys (round_key_0 through round_key_10) from the user-supplied 128-bit master key using the AES key schedule algorithm. Importantly, this happens in parallel and combinationally—all round keys are available immediately without waiting for sequential computation. The datapath uses these round keys directly:

- Round 0 uses round_key_0 (the original master key) in AddRoundKey
- Rounds 1–9 use round_key_1 through round_key_9 in their AddRoundKey steps
- Round 10 uses round_key_10 in its final AddRoundKey

By precomputing, the pipeline avoids latency stalls waiting for round key generation, maximizing throughput. A new block is processed every 10 clock cycles (after stage 10 completes).

Pipeline Latches & Address Tracking

All 10 latches propagate not just the encrypted data but also the address of the plaintext block. This is essential: the write controller must know where in external memory to write each ciphertext result.

Latches Track:
- 128-bit data (plaintext → intermediate states → ciphertext)
- 128-bit address (input block address passed through all stages unchanged)
- 1-bit valid signal (valid_in at stage 0, valid_out at stage 10, intermediate valid bits in between)

Stall Propagation: When the global stall signal is asserted, all latches freeze simultaneously. The valid signals

are also held, preventing any state change. This ensures data consistency and prevents loss or corruption.

Memory Controllers & AXI Transactions

Read Controller (Plaintext Fetcher)

Purpose: Orchestrates AXI4-Lite read transactions to fetch plaintext from external memory and queue it into the read_mem FIFO.

State Machine: The read controller uses two independent state machines:
1. **AR (Address Read) State Machine:** Manages the AXI address phase.
• Starts in ar_idle. On START, transitions to ar_run.
• In ar_run, issues ARVALID every clock (if ARREADY is high, read_mem is not full, and not stalled).
• Increments req_addr by 4 bytes (one 32-bit word) after each successful transaction.
• Returns to ar_idle when req_addr >= end_address.
2. **R (Data Read) State Machine:** Manages the AXI data phase.
• Starts in r_idle. On START, transitions to r_run.
• In r_run, sets RREADY high (if read_mem not full and not stalled).
• When RVALID arrives (and RREADY is high), captures RDATA and writes to read_mem.
• Returns to r_idle when resp_addr >= end_address.

Decoupled Address & Data: The two state machines run independently, allowing addresses to be pipelined ahead of data (a feature of AXI4-Lite). The address pointer (req_addr) can advance even if the data hasn't arrived yet, improving throughput with pipelined memory systems.

Error Handling: Read responses (RRESP) are checked; only OKAY (2'b00) responses are written to the FIFO.

Write Controller (Ciphertext Writer)

Purpose: Orchestrates AXI4-Lite write transactions to store encrypted blocks back to external memory, draining the write_mem FIFO.

State Machine: The write controller uses a single unified state machine:
• Starts in aw_idle. On START, transitions to aw_run.
• In aw_idle: Sets rd_en high to read one entry from write_mem (if not empty). Sets both AWVALID and WVALID high, preparing the AW (write address) and W (write data) channels.
• In aw_run: Monitors both AWVALID/AWREADY (address channel) and WVALID/WREADY (data channel). Both must complete for a full write transaction (aw_done && w_done).
• When both channels succeed: Reads the next entry from write_mem (if not empty) and continues iterating.
• Asserts stall to the rest of the system if either channel is not ready, preventing pipeline overflow.
• Returns to aw_idle when all results have been written (write_mem empty) and done signal is asserted.

Back-Pressure & Stall: If the external memory (slave) is slow to accept write data, the write controller stalls the entire system, preventing read_mem from filling with unprocessable data.

Error Detection: The write controller monitors BRESP (write response) for errors. If any response is not OKAY (2'b00), the error signal is asserted and STATUS becomes ERROR.

Performance & Throughput Analysis

Pipeline Latency: From the time plaintext enters the pipeline (valid_in high at stage 0) to the time ciphertext emerges (valid_out high at stage 10), exactly 10 clock cycles elapse.

Throughput: Under ideal conditions (no memory stalls), one 128-bit block is output every 10 clock cycles. This translates to 1 block/10 cycles = 0.1 blocks/cycle or roughly 12.8 Gbps at a typical AES implementation frequency (assume 400 MHz).

Stall Impact: If external memory is slow (e.g., latency > 10 cycles per read/write), the FIFOs will eventually fill or empty, causing system-wide stalls. The stall signal throttles both plaintext reads and encryption output, reducing throughput to match external memory bandwidth.

FIFO Sizing: The 4-entry FIFOs were chosen as a balance: small enough to fit in modest area, large enough to tolerate a few clock cycles of memory latency without stalling the pipeline. Larger FIFOs would improve tolerance of slower memory but increase area and complexity.

Design Decisions & Trade-offs

Precomputed Round Keys:

Round keys are computed combinationally at startup. Trade-off: silicon area (11 copies of 128 bits) versus latency. The decision prioritizes throughput; in resource-constrained designs, keys could be computed on-the-fly.

10-Stage Unrolled Pipeline:

All 10 AES rounds are unrolled into separate pipeline stages. Trade-off: high area (10 × SubBytes, ShiftRows, MixColumns modules) versus high throughput. An alternative would be a single AES round stage that repeats 10 times, using ~1/10 the area but at 1/10 throughput.

Dual-Port FIFOs:

Read_mem uses 4-entry FIFO with write port (32-bit, from read_controller) and read port (128-bit, to pipeline). Trade-off: Asymmetric granularity allows fine-grained external memory control but requires careful FIFO logic. A symmetric design would simplify but reduce flexibility.

Global Stall Signal:

Write controller asserts stall to halt all domains. Trade-off: simple (one signal) but coarse-grained. A more refined design might use local back-pressure signals per module, but the global stall is sufficient for correctness.

Sequential Memory Addressing:

The accelerator reads and writes contiguous 32-bit words from start_address to end_address. Trade-off: simple address calculation but inflexible. Modern accelerators might support scatter/gather addressing.

Synchronous Control:

All control signals (START, DONE, stall) are sampled/updated on clock edges. Trade-off: deterministic behavior but slightly higher latency. Asynchronous signaling would reduce latency but complicate verification.

Integration & Verification Approach

The AES accelerator is designed to integrate into a larger SoC with a CPU, memory bus, and interconnect. The AXI4-Lite interfaces (both slave for control and master for memory) are standard and compatible with common open-source and commercial interconnects.

Testbench Strategy: Simulation provides a 10-entry NIST test vector suite with known plaintext-ciphertext pairs. Each test vector exercises the full pipeline from CPU register writes through memory transactions. The testbench verifies:
• Correct ciphertext output for known plaintext/key
• Proper status transitions (IDLE → BUSY → DONE)
• Correct handling of start/end addresses
• FIFO full/empty logic under various timing scenarios
• Error responses from memory (simulated via BRESP injection)

Synthesis Considerations: The design synthesizes cleanly to standard-cell libraries. The precomputed round keys and S-box lookup tables are the main area contributors. Clock frequency is typically limited by the pipeline stages (each containing logic depth for SubBytes, ShiftRows, MixColumns, AddRoundKey). Timing analysis should focus on the datapath's critical path and the FIFO pointer comparison logic.

Future Enhancements & Variants

• Support for AES-192 and AES-256 by extending key size and adding more rounds.

• Configurable block size (e.g., GCM or CTR modes) instead of ECB.

• Decryption path (InvShiftRows, InvSubBytes, InvMixColumns) for bidirectional operation.

• Scatter/gather addressing to encrypt non-contiguous memory regions.

• Burst support (AXI4 instead of AXI4-Lite) for higher memory bandwidth.

• Interrupt generation on completion (instead of polling STATUS).

• Hardware reset and key revocation features for security.

• Integration of a TRNG (True Random Number Generator) for IV generation in modes like CBC or CTR.

Summary: Key Takeaways

The AES-128 accelerator demonstrates a well-structured approach to hardware encryption:

1. Clean Separation of Concerns: Control (register interface), key management (key scheduler), data transformation (pipeline), and memory management (dual-FIFO architecture) are logically separated into independent modules.

2. Efficient Pipelined Execution: An unrolled 10-stage pipeline delivers high throughput (one block per 10 cycles) while precomputed round keys eliminate key scheduling latency.

3. Intelligent Memory Buffering: Asymmetric dual-FIFO architecture (4-word writes, 1-word reads on write side) balances area, latency, and flexibility. Stall signals provide global flow control without complex per-module handshakes.

4. Standard Interface Compliance: AXI4-Lite master and slave ports make the accelerator compatible with standard SoC integration flows and eliminates custom glue logic.

5. Practical Trade-offs: Every design choice (precomputation vs. on-the-fly, unrolled vs. iterated pipeline, FIFO sizing, stall granularity) reflects a deliberate trade-off between performance, area, and complexity. These decisions are documented and defensible for the target use case (edge encryption, embedded systems).

By studying this design, hardware engineers learn how to architect high-performance crypto accelerators that are both efficient (in silicon and power) and practical (in verification and integration).

For implementation details and register-level traces, refer to the individual module source files and testbench. This document provides the architectural blueprint; the code offers granular verification and simulation artifacts.